

به نام خدا

طراحی کامپایلر

آرش شفیعی



- کامپایلرها: اصول، روش‌ها، و ابزارها از آهو و همکاران¹
- طراحی کامپایلر مدرن از گرون و همکاران²
- پیاده‌سازی کامپایلر مدرن در جاوا از اپل³

¹ Compilers: Principles, Techniques, and Tools, by Aho et al.

² Modern Compiler Design, by Grune et al.

³ Modern Compiler Implementation in Java, by Appel

مقدمه

- یک زبان ¹ مجموعه‌ای است که از تعدادی رشته (جمله) ² تشکیل شده است.
- یک رشته دنباله‌ای است از تعدادی حرف (کاراکتر) ³ که متعلق به یک الفبا ⁴ هستند.
- مثلاً زبان $L(ab^*) = \{a, ab, abb, \dots\}$ مجموعه‌ای است از همه رشته‌هایی که در ابتدای آنها حرف a و پس از آن تعدادی b قرار گرفته است. رشته‌های این زبان متعلقند به الفبای $\Sigma = \{a, b\}$.

¹ language

² string (sentence)

³ character

⁴ alphabet

- به ازای یک رشته و یک زبان داده شده می‌توانیم تعیین کنیم آیا آن رشته عضو به زبان است یا خیر. مثلاً رشته $abbb$ عضو زبان $L(ab^*)$ است، اما رشته $abba$ عضو این زبان نیست.
- دو سازوکار برای بررسی یک عضویت یک رشته به یک زبان وجود دارد: تشخیص دهنده‌ها¹ و تولیدکننده‌ها².
- تشخیص دهنده‌ها ماشین‌هایی هستند که یک رشته را حرف به حرف تا انتهای رشته دریافت می‌کند و تشخیص می‌دهند آیا آن رشته عضو زبان است یا خیر. تولیدکننده‌ها، که گرامر³ نامیده می‌شوند، همه رشته‌های ممکن از یک زبان را تولید می‌کنند.
- به ازای یک رشته و یک گرامر داده شده، می‌توانیم بررسی کنیم آیا آن رشته توسط گرامر قابل تولید است یا خیر. به الگوریتمی که به ازای رشته دریافت شده، قابلیت تولید شدن آن توسط یک گرامر را بررسی می‌کند، تجزیه‌کننده⁴ گفته می‌شود.

¹ recognizers

² generators

³ grammar

⁴ parser

- در نظریه زبان‌ها و ماشین‌ها، سلسله مراتب زبان‌های صوری (قراردادی) را مطالعه کردیم.
- چامسکی، یکی از زبان‌شناسان سرشناس، زبان‌های صوری را به چهار دسته تقسیم کرده است: زبان‌های منظم، زبان‌های مستقل از متن، زبان‌های حساس به متن، و زبان‌های شمارش‌پذیر بازگشتی.
- برای هر دسته از زبان‌ها، دسته‌ای از ماشین‌ها وجود دارد که عضویت یک رشته به آن زبان را بررسی می‌کند. همچنین برای هر دسته از زبان‌ها، دسته‌ای از گرامرها وجود دارد که می‌تواند همه رشته‌های موجود در آن زبان‌ها را تولید کند.

مخفف	زبان	گرامر	ماشین
REG نوع ۳	منظم	$A \rightarrow aB, A \rightarrow Ba, A \rightarrow a$ $A, B \in V, a \in T^*$	متناهی قطعی و متناهی غیرقطعی
CF نوع ۲	مستقل از متن	$A \rightarrow \alpha$ $A \in V, \alpha \in (V \cup T)^*$	پشته‌ای
CS نوع ۱	حساس به متن	$x \rightarrow y$ $x, y \in (V \cup T)^+, y \geq x $	کراندار خطی
RE نوع ۰	شمارش‌پذیر بازگشتی	$x \rightarrow y$ $x \in (V \cup T)^+, y \in (V \cup T)^*$	تورینگ

- زبان‌های منظم توسط گرامر منظم قابل تولید هستند. این گرامرها می‌توانند چپ خطی یا راست خطی باشند. همچنین ماشین‌های متناهی شامل متناهی قطعی و متناهی غیر قطعی می‌توانند تعلق یک رشته به یک زبان منظم را بررسی کنند.
- زبان‌های مستقل از متن توسط گرامرهای مستقل از متن توصیف می‌شوند. برای تجزیه یک رشته توسط یک گرامر مستقل از متن الگوریتم‌های متنوعی وجود دارند. یکی از این الگوریتم‌ها، الگوریتم سی‌وای کی¹ است که یک الگوریتم از نوع برنامه‌ریزی پویا است و به ازای رشته داده شده با طول n در زمان $O(n^3)$ تعیین می‌کند آیا رشته توسط گرامر قابل تجزیه است یا خیر.

¹ Cocke-Younger-Kasami (CYK) algorithm

- نظریه زبان‌ها و ماشین‌ها دو کاربرد مهم دارد: نظریه محاسبات و طراحی کامپایلر.

- در نظریه محاسبات، هر مسئله توسط یک زبان مدل‌سازی می‌شود. یک مسئله می‌پرسد آیا الگوریتمی وجود دارد که یک مسئله محاسباتی معین با شرح معین را حل کند و به ازای ورودی‌های داده شده، خروجی‌های شرح شده توسط مسئله را تولید کند یا خیر.
- در نظریه محاسبات، هر مسئله را به صورت یک زبان در نظر می‌گیریم که تشکیل شده است از همهٔ جواب‌ها مسئله. مثلاً مسئله بررسی اول بودن یک عدد را در نظر بگیرید. در این مسئله می‌پرسیم آیا یک عدد اول است یا خیر. زبان معادل این مسئله، زبانی است که تشکیل شده است از همهٔ اعداد اول. حال به ازای یک عدد داده شده باید بررسی کنیم آیا آن عدد متعلق به آن زبان است یا خیر. برای چنین زبانی می‌توانیم یک ماشین تورینگ طراحی کنیم. اگر ماشین تورینگ برای یک زبان وجود داشته باشد، برای مسئله معادل آن نیز یک الگوریتم وجود دارد. پس برای یک مسئله یک الگوریتم وجود دارد اگر و تنها اگر برای زبان معادل آن ماشین تورینگ وجود داشته باشد.
- در نظریه محاسبات بررسی می‌کنیم چه مسائلی محاسبه‌پذیرند و یا به عبارت دیگر برای چه مسائلی ماشین تورینگ وجود دارد. در نظریهٔ محاسبات روش‌هایی برای اثبات محاسبه‌پذیری مسائل وجود دارد.

- در طراحی کامپایلر، یک زبان برنامه‌نویسی را به صورت یک زبان صوری در نظر می‌گیریم و از نظریه محاسبات برای توصیف و تجزیه آن استفاده می‌کنیم.
- برای مثال زبان سی مجموعه‌ای است از همه رشته‌هایی که توسط زبان سی می‌توانند نوشته شوند. بنابراین
$$L(C) = \{"void\ main()\ \{\}\", "int\ main()\ \{\ return\ 0;\}\", \dots\}$$
- همه زبان‌های برنامه‌نویسی موجود را می‌توان توسط گرامر مستقل از متن توصیف کرد. پس به ازای یک رشته داده شده (که یک برنامه نوشته شده در آن زبان است)، می‌توان رشته را توسط گرامر مستقل از متن آن زبان تجزیه کرد و بررسی کرد که آیا آن برنامه در آن زبان یک برنامه معتبر است یا خیر. ماشینی که یک برنامه در یک برنامه را دریافت کرده و تعلق آن برنامه به زبان را بررسی می‌کند، قسمتی از کامپایلر آن زبان است.
- بنابراین می‌توان برای $L(C)$ نیز یک گرامر مستقل از متن توصیف کرد. حال می‌توانیم توسط یک تجزیه کننده، یک برنامه داده شده در زبان سی، توسط گرامر زبان سی، بررسی کنیم آیا آن برنامه معتبر است یا خیر.

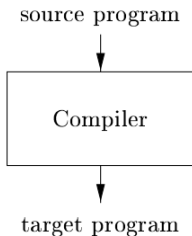
- گرچه الگوریتم سی‌وای‌کی می‌تواند در زمان $O(n^3)$ عضویت رشته‌ای با طول n را به یک زبان مستقل از متن بررسی کند، اما این زمان برای کامپایلرها در عمل به اندازه کافی سریع نیست.
- در علم کامپایلر الگوریتم‌های تجزیه‌کننده سریع‌تری ابداع شده‌اند که می‌توانند در زمان خطی یعنی $O(n)$ یک رشته با طول n را تجزیه کنند.
- البته این الگوریتم‌ها نمی‌توانند همه گرامرهای مستقل از متن را تجزیه کنند، بلکه تنها می‌توانند زیرمجموعه‌ای از زبان‌های مستقل از متن را تجزیه کنند. اما تقریباً همه زبان‌های برنامه‌نویسی در این زیرمجموعه قرار می‌گیرند.
- در این درس، ابتدا بررسی می‌کنیم چگونه برای یک زبان برنامه‌نویسی گرامر طراحی کنیم. سپس با الگوریتم‌های سریعی برای تجزیه برنامه‌های یک زبان ارائه می‌کنیم.

- زبان برنامه‌نویسی یک روش نشانه‌گذاری¹ برای توصیف محاسبات است.
- برای این که محاسبات توصیف شده توسط یک ماشین قابل فهم و اجرا باشد، محاسبات توصیف شده توسط یک زبان برنامه‌نویسی باید به زبان ماشین ترجمه شود.
- برنامه‌ای که محاسبات توصیف شده در یک زبان برنامه‌نویسی را به زبان ماشین ترجمه می‌کند کامپایلر² نامیده می‌شود.
- در مطالعه و پیاده‌سازی کامپایلرها بسیاری از مباحث زبان‌های برنامه‌نویسی، معماری ماشین‌ها، نظریه زبان‌ها و ماشین‌ها، الگوریتم‌ها و مهندسی نرم‌افزار مورد استفاده قرار می‌گیرند.
- در این فصل به معرفی اجمالی کامپایلرها و ساختار کلی آنها می‌پردازیم.

¹ notation

² compile

- کامپایلر برنامه‌ای است که یک برنامه در یک زبان برنامه‌نویسی را که زبان مبدأ¹ نامیده می‌شود، دریافت می‌کند و آن را به معادل آن در یک زبان دیگر که زبان مقصد² نامیده می‌شود ترجمه می‌کند.

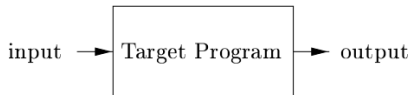


- کامپایلر همچنین خطاهایی را که در برنامه مبدأ وجود دارند را در حین ترجمه تشخیص می‌دهد.

¹ source language

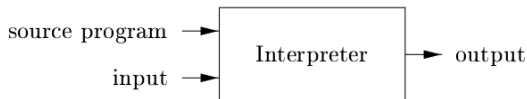
² target language

- وقتی زبان مقصد برنامه‌ای به زبان ماشین باشد، آنگاه کاربر می‌تواند ورودی خود را جهت انجام محاسبات به برنامه مقصد ارسال کند و نتیجه محاسبات را دریافت کند.



پردازشگرهای زبان

- یک مفسر¹ نوع دیگری از پردازشگرهای زبان است. یک مفسر به جای تولید برنامه در زبان مقصد، مستقیماً برنامه مبدأ و ورودی کاربر را دریافت می‌کند و نتیجه محاسبات را به عنوان خروجی تولید می‌کند.



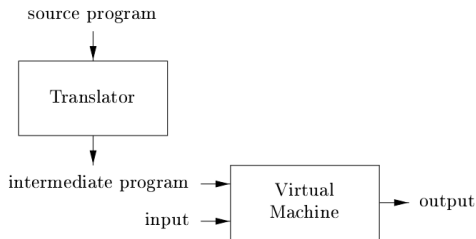
- برنامه تولید شده در زبان ماشین توسط کامپایلر معمولاً سریع‌تر از مفسر محاسبات را انجام می‌دهد.
- از طرف دیگر مزیت مفسر این است که کد نوشته شده بر روی هر ماشینی قابل اجراست و همچنین بهتر از کامپایلر می‌تواند به تشخیص خطاها توسط برنامه نویس کمک کند زیرا دستورات در آن یک‌به‌یک اجرا می‌شوند.

¹ interpreter

- مشکل اصلی پیاده‌سازی یک زبان توسط کامپایلر این است که کد تولید شده تنها بر روی ماشینی که برای آن کامپایل شده است قابل اجرا است. مزیت آن این است که کد کامپایل شده سرعت اجرای بالایی دارد. از طرف دیگر مشکل اصلی مفسر سرعت اجرای پایین آن است و مزیت آن این است که کد نوشته شده بر روی هر ماشینی قابل اجراست.

پردازشگرهای زبان

- از آنجایی که کامپایل و تفسیر هر کدام مزیت‌های خاص خود را دارند، برخی از پردازشگرهای زبان از یک روش ترکیبی استفاده می‌کنند.
- پردازشگرهای زبان جاوا کامپایل و تفسیر را ترکیب می‌کنند. یک برنامه جاوا ابتدا به یک برنامه میانی به نام بایت‌کد¹ کامپایل می‌شود. سپس بایت‌کد تولید شده توسط یک مفسر به نام ماشین مجازی جاوا² تفسیر می‌شود.



¹ bytecode

² Java virtual machine

- مزیت این روش این است که بایت‌کد کامپایل شده بر روی یک ماشین می‌تواند بر روی یک ماشین دیگر تفسیر شود و در عین حال سرعت اجرای آن نیز بهتر از روش تفسیر است.
- از آنجایی که اجرای برنامه توسط پردازشگرهای ترکیبی زبان نسبت به اجرای برنامه به زبان ماشین سرعت پایین‌تری دارد، کامپایلرهایی به نام کامپایلرهای درجا¹ وجود دارند که بایت‌کد را به کد زبان ماشین ترجمه می‌کنند.

¹ just-in-time

- برای تولید برنامه قابل اجرا در زبان مقصد، علاوه بر یک کامپایلر، به برنامه‌های دیگری نیز نیاز است.
- یک برنامه مبدأ می‌تواند به ماژول‌های مختلف در چند فایل جداگانه ذخیره شده باشد. وظیفه جمع‌آوری برنامه مبدأ از فایل‌های مختلف به عهده پیش‌پردازنده¹ است. همچنین یک پیش‌پردازنده می‌تواند دستوراتی را که به عنوان مخفف به جای دستورات اصلی به کار رفته‌اند (ماکروها²) را با دستورات اصلی جایگزین کند.
- پس از پردازش پیش‌پردازنده، برنامه تبدیل شده به کامپایلر ارسال می‌شود. کامپایلر، یک برنامه به زبان اسمبلی³ به عنوان خروجی تولید می‌کند، زیرا تولید کد اسمبلی ساده‌تر از تولید کد به زبان ماشین است.
- برنامه اسمبلی به یک اسمبلر⁴ داده می‌شود و کد برنامه به زبان ماشین تولید می‌شود.

¹ preprocessor

² macros

³ assembly

⁴ assembler

- برنامه‌های بزرگ معمولاً به صورت قطعات جداگانه کامپایل می‌شوند. به همین جهت نیاز است قطعات کامپایل شده با یکدیگر پیوند داده شوند. پیوند برنامه‌ها توسط پیوند دهنده یا لینکر¹ انجام می‌شود.
- در پایان برنامه اجرایی تولید شده توسط لینکر به وسیله لودر² در حافظه قرار می‌گیرد.

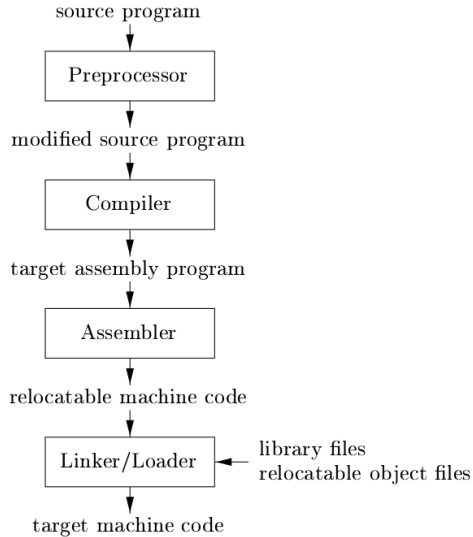
¹ linker

² loader

- در صورتی که یک برنامه بخواهد از یک کتابخانه ایستا¹ استفاده کند، لینکر آن را به برنامه پیوند می‌دهد و کد اجرایی تولید شده از الحاق برنامه کامپایل شده و کتابخانه کامپایل شده به دست می‌آید.
- اما در صورتی که یک برنامه بخواهد از یک کتابخانه پویا² استفاده کند، لودر در زمان اجرای برنامه کد ماشین کتابخانه پویا را در حافظه قرار می‌دهد.
- مزیت استفاده از کتابخانه ایستا این است که کد برنامه می‌تواند به صورت مستقل اجرا شود و عیب آن این است که کد اجرایی تولید شده حجم بیشتری اشغال می‌کند. مزیت استفاده از کتابخانه پویا این است که اگر کتابخانه به طور مجزا به روز رسانی شود، نیاز به کامپایل مجدد برنامه وجود ندارد. همچنین کد کامپایل شده برنامه حجم کمتری اشغال می‌کند. همچنین کتابخانه‌های پویا را می‌توان بین چند برنامه به اشتراک گذاشت.

¹ static library

² dynamic library



- ساختار داخلی کامپایلر را می‌توان به دو بخش تحلیل یا آنالیز¹ و بخش ترکیب یا سنتز² تقسیم کرد.
- بخش تحلیل برنامه مبدأ را بر اساس گرامر آن ساختار بندی و تقسیم‌بندی می‌کند. سپس از این تحلیل برای تولید یک کد میانی معادل برنامه مبدأ استفاده می‌کند. اگر در تحلیل کد مبدأ، کامپایلر یک خطای نحوی³ یا معنایی⁴ تشخیص دهد، باید پیام‌های خطای مناسب تولید کند تا کاربر، برنامه را تصحیح کند. در بخش تحلیل، یک جدول علائم⁵ نیز تهیه می‌شود که ساختار داده‌ای برای نگهداری اطلاعات درمورد برنامه است. این جدول علائم نیز به بخش سنتز کامپایلر ارسال می‌شود.
- بخش سنتز با استفاده از اطلاعات دریافت شده از بخش تحلیل، یعنی نمایش میانی ساختار برنامه⁶ (کد میانی) و جدول علائم، برنامه مقصد را تولید می‌کند.

¹ analysis

² synthesis

³ syntax error

⁴ semantic error

⁵ symbol table

⁶ intermediate representation

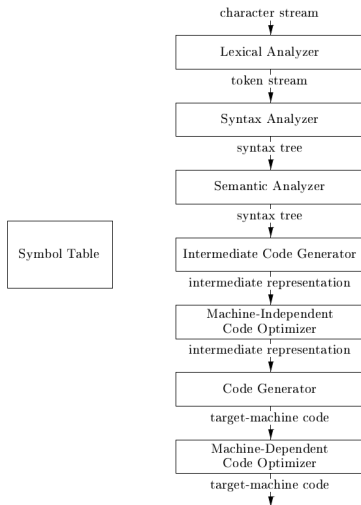
- بخش تحلیل یا آنالیز، بخش ¹ ساخت یا سمت کاربر کامپایلر و بخش ترکیب یا سنتز، بخش زیرساخت ² یا سمت ماشین کامپایلر نیز نامیده می‌شود.
- یک کامپایلر واحدی به نام بهینه‌ساز کد را نیز شامل می‌شود که ساختار دریافت شده از بخش تحلیل را دریافت کرده و آن را بهینه می‌کند تا کد مقصد بهتری تولید شود.

¹ front end

² back end

ساختار کامپایلر

- یک کامپایلر از قسمت‌های زیر تشکیل شده است. جدول علائم با همه قسمت‌ها در ارتباط است.



- اولین مرحله در فرایند کامپایل تحلیل لغوی یا واژگانی¹ یا اسکنر² نامیده می‌شود.
- تحلیل‌گر لغوی جریانی از کاراکترها را که برنامه مبدأ را تشکیل می‌دهند دریافت می‌کند و آنها را به دنباله‌های معناداری به نام واژه³ تبدیل می‌کند.
- به ازای هر واژه، تحلیل‌گر لغوی، یک توکن⁴ به صورت یک زوج شامل نام توکن⁵ و مقدار ویژگی⁶ آن تولید می‌کند. `<token-name, attribute-value>`
- نام توکن در واقع مفهوم انتزاعی یا نوع واژه است و ویژگی توکن می‌تواند خود واژه باشد و یا به مکان واژه در جدول علائم اشاره کند. توکن‌ها به واحد بعدی کامپایلر یعنی تحلیل‌گر نحوی ارسال می‌شوند.

¹ lexical analysis

² scanner

³ lexeme

⁴ token

⁵ token name

⁶ attribute value

- برای مثال فرض کنید برنامه مبدأ یک دستور انتساب مقدار به صورت زیر باشد.
$$\text{position} = \text{initial} + \text{rate} * 60$$
- حروف یا کاراکترهای این دستور را می‌توان دسته‌بندی کرد و واژه‌های زیر را تولید کرد.
- ۱- واژه position به توکن $\langle \text{id}, 1 \rangle$ نگاشت می‌شود، جایی که id نوعی انتزاعی برای مفهوم شناسه¹ است و 1 به مکان واژه position در جدول علائم اشاره می‌کند. در جدول علائم در مکان واژه position ، نوع و نام این متغیر ذخیره شده است.
- ۲- نماد انتساب $=$ واژه‌ای است که به توکن $\langle = \rangle$ نگاشت می‌شود. این توکن هیچ مقدار ویژگی ندارد. می‌توانستیم به جای $\langle = \rangle$ از توکن $\langle \text{assign} \rangle$ نیز استفاده کنیم ولی برای سهولت نشانه‌گذاری از توکن $\langle = \rangle$ استفاده می‌کنیم.
- ۳- واژه initial به توکن $\langle \text{id}, 2 \rangle$ نگاشت می‌شود.

¹ identifier

تحلیل لغوی

- ۴- نماد جمع + به توکن $\langle + \rangle$ نگاشت می‌شود.
- ۵- واژه rate به توکن $\langle id, 3 \rangle$ نگاشت می‌شود.
- ۶- نماد ضرب * واژه‌ای است که به توکن $\langle * \rangle$ نگاشت می‌شود.
- ۷- عدد 60 واژه‌ای است که به توکن $\langle 60 \rangle$ نگاشت می‌شود.
- از فاصله‌های خالی بین واژگان در تحلیل‌گر واژگانی چشم‌پوشی می‌شود.
- بنابراین دستور قبلی پس از تحلیل واژگانی به صورت زیر تبدیل می‌شود.
$$\langle id, 1 \rangle \langle = \rangle \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle \langle 60 \rangle$$

```
position = initial + rate * 60
```

```
graph TD; A["position = initial + rate * 60"] --> B[Lexical Analyzer]; B --> C["<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>"]
```

- دومین گام کامپایلر تحلیل نحوی¹ یا تجزیه² است. تجزیه کننده³ از توکن‌های تهیه شده توسط تحلیل‌گر لغوی استفاده می‌کند و یک نمایش درختی از ساختار گرامری جریان یا رشته ورودی می‌سازد.
- به ساختار تولید شده توسط تحلیل‌گر نحوی، درخت تجزیه⁴ گفته می‌شود.
- گاهی کلمه درخت نحوی⁵ به عنوان مترادف درخت تجزیه به کار برده می‌شود، با اینکه تفاوت‌های اندکی بین درخت تجزیه و درخت نحوی وجود دارد.

¹ syntax analysis

² parsing

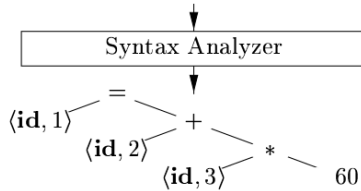
³ parser

⁴ parse tree

⁵ syntax tree

- درخت تجزیه برای عبارت $\text{position} = \text{initial} + \text{rate} * 60$ در زیر نمایش داده شده است. در این درخت تجزیه ترتیب اعمال عملگرها تعیین می‌شود.

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$



- توسط این درخت، عملگر ضرب، سپس عملگر جمع و در پایان عملگر انتساب ارزیابی می‌شوند.

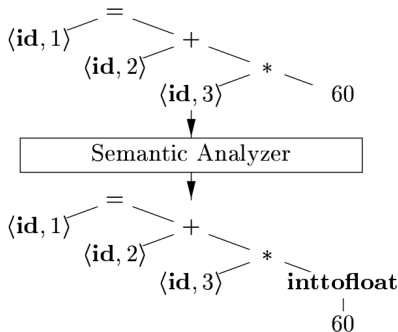
- تحلیل‌گر معنایی¹ درخت تجزیه را به همراه جدول علائم دریافت می‌کند و درستی معنایی برنامهٔ مبدأ را بررسی می‌کند.
- یک قسمت مهم تحلیل معنا، بررسی نوع² است، که توسط آن کامپایلر بررسی می‌کند هر عملگر با عملوندهای آن مطابقت داشته باشند.
- تحلیل‌گر معنایی درخت تجزیه یا درخت نحوی را دریافت می‌کند و بر روی آن عملیات تحلیل معنا انجام می‌دهد و یک درخت نحوی دیگر تولید می‌کند.

¹ semantic analyzer

² type checking

- برای مثال اگر در یک عبارت انتساب مقدار، نوع متغیرها در دو طرف عملگر همخوانی نداشته باشند، کامپایلر باید پیام خطا صادر کند. این بررسی در تحلیل‌گر معنایی انجام می‌شود.
- در یک زبان ممکن است برخی از تبدیل نوع‌ها مجاز باشند. برای مثال یک عملگر حسابی دوگانی می‌تواند یک جفت عدد صحیح یا یک جفت عدد اعشاری را به عنوان عملوند دریافت کند. اگر یک عملوند دوگانی یک عدد صحیح و یک عدد اعشاری دریافت کرد، کامپایلر ممکن است عدد صحیح را به اعشاری تبدیل کند. این عملیات نیز در تحلیل‌گر معنایی انجام می‌شود.

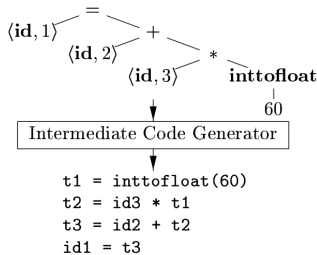
- در مثال قبل rate یک عدد اعشاری است بنابراین عملوند دوم عملگر ضرب یعنی عدد 60 توسط کامپایلر به یک عدد اعشاری تبدیل می‌شود. برای این تبدیل که به صورت ضمنی انجام می‌شود از عملگر `inttofloat` استفاده شده است.



- در فرایند ترجمه یک برنامه مبدأ به یک برنامه مقصد، کامپایلر ممکن است یک یا چند نمایش از کد مبدأ تولید کند.
- برای مثال درخت نحوی یک نمایش میانی از کد مبدأ است که در تحلیل معنایی تولید می‌شود.
- پس از تحلیل معنایی، معمولاً کامپایلرها یک نمایش میانی شبیه به کد ماشین تولید می‌کنند که همان برنامه در یک زبان میانی است.
- این کد میانی باید دو ویژگی داشته باشد. اول اینکه کد میانی باید به آسانی توسط برنامه مبدأ قابل تولید باشد و دوم اینکه کد میانی باید بتواند به سهولت به کد مقصد تبدیل شود.

تولید کد میانی

- برای مثال یک کد میانی به نام کد سه آدرسی¹ که شبیه کد اسمبلی است، در مرحله تولید کد میانی تولید می‌شود. کد میانی برای دستور قبل به صورت زیر است :



- در این کد میانی در سمت راست عبارت انتساب حداکثر یک عملگر وجود دارد، بنابراین ترتیب اعمال عملگرها مشخص است. همچنین برای مقادیر میانی نام‌های موقت تعریف می‌شود. هر عبارت در این کد میانی حداکثر سه عملوند دارد و به همین دلیل به آن کد سه آدرسی گفته می‌شود.

¹ three-address code

- مرحله بهینه‌سازی مستقل از کد ماشین¹، کد میانی تولید شده را بهینه‌سازی می‌کند تا کد بهتری در پایان برای ماشین مقصد تولید شود. کد بهتر معمولاً کدی است که سریع‌تر اجرا می‌شود، اما می‌توانیم از معیارهای دیگری نیز برای ارزیابی کد بهتر استفاده کنیم. برای مثال معیار ارزیابی می‌تواند کوتاه‌تر بودن کد یا میزان مصرف انرژی کد باشد.

¹ machine-independent code-optimization

- در مثال قبل بهینه‌ساز کد مستقیماً عدد صحیح 60 را به عدد اعشاری 60.0 تبدیل می‌کند و دستور `inttofloat` را حذف می‌کند. علاوه بر آن عبارت انتساب `id1 = t3` تنها یک مقدار را منتقل می‌کند، بنابراین بهینه‌ساز کد متغیر `t3` را حذف می‌کند.

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```



Code Optimizer



```
t1 = id3 * 60.0
id1 = id2 + t1
```

- برخی از روش‌های بهینه‌سازی کد وجود دارند که با کاستن سرعت کامپایل به میزان اندکی، کد مقصد را به میزان قابل توجهی بهبود می‌دهند.
- برخی دیگر از روش‌های بهینه‌سازی پیچیده‌تر نیز وجود دارند که به زمان بیشتری برای اجرا نیاز دارند که در کامپایلرهای بهینه‌ساز استفاده می‌شوند.

- در مرحله تولید کد¹، کد میانی معادل برنامه ورودی دریافت می‌شود و به یک برنامه در زبان مقصد تبدیل می‌شود. اگر زبان مقصد، زبان اسمبلی مختص به یک ماشین باشد، آنگاه از رجیسترها و قابلیت‌های ماشین مقصد برای تبدیل کد استفاده می‌شود. در مرحله تولید کد، استفادهٔ بهینه از رجیستر اهمیت زیادی پیدا می‌کند.

¹ code generation

- برای مثال با استفاده از رجیسترهای R1 و R2 در مثال قبل کد زیر تولید می‌شود.

```
t1 = id3 * 60.0
id1 = id2 + t1
```



Code Generator

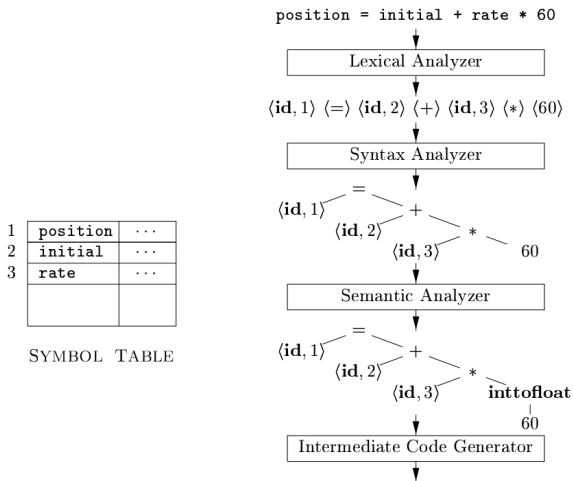


```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

- حرف F در این دستورات نشان می‌دهد که محاسبات بر روی اعداد اعشاری انجام می‌شود. دستور LD برای کپی کردن مقدار از حافظه در رجیستر و ST برای ذخیره کردن مقدار یک رجیستر در یک فضای حافظه استفاده شده است. دستور ADD برای جمع مقدار دو رجیستر و دستور MUL برای ضرب مقدار دو رجیستر استفاده شده است.

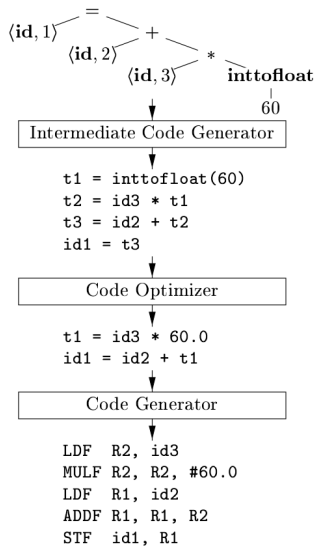
- یکی از وظایف کامپایلر نگهداری جدول علائم¹ برای ذخیره‌سازی نام متغیرها و نوع ویژگی‌های دیگر آنهاست که در طول برنامه از آن استفاده می‌شود.
- این ویژگی‌ها اطلاعاتی را نگهداری می‌کنند که همهٔ بخش‌های کامپایلر از آنها استفاده می‌کنند. برای مثال برای یک متغیر، نام، نوع، و حوزه تعریف متغیر اطلاعات مهمی هستند و برای یک تابع، نام، تعداد و نوع پارامترها و نوع فراخوانی پارامترها (فراخوانی با مقدار یا ارجاع یا غیره) و نوع دادهٔ بازگردانده شده از تابع اطلاعاتی هستند که می‌توانند در جدول علائم ذخیره شوند.

¹ symbol table



1	position	...
2	initial	...
3	rate	...

SYMBOL TABLE



دسته‌بندی اجزای کامپایلر

- مراحل مختلف و اجزای متفاوت یک کامپایلر را می‌توان به دسته‌هایی تقسیم کرد. برای مثال می‌توان اجزای مقدماتی از جمله تحلیل‌گر لغوی، تحلیل‌گر نحوی، تحلیل‌گر معنایی و تولیدکننده کد میانی را می‌توان در یک گروه قرار داد و بهینه‌ساز کد و تولیدکننده کد را در یک گروه دیگر.
- بدین ترتیب یک کامپایلر به طور کلی از دو قسمت سمت کاربر یا بخش روساخت¹ و سمت ماشین یا بخش زیرساخت² تشکیل شده است.
- بخش زیرساخت بسته به زبان ماشین مقصد متفاوت خواهد بود و بنابراین به ازای ماشین‌های متفاوت بخش‌های زیرساخت متنوعی وجود خواهند داشت. اما بخش روساخت به ازای یک زبان ورودی همیشه یکسان است.
- با جداسازی این دو بخش می‌توان کامپایلری عرضه کرد که بخش زیرساخت آن برای ماشین‌های جدید قابل تعویض باشد.

¹ front-end

² back-end

- سازندگان کامپایلر مانند همه توسعه‌دهندگان نرم‌افزار از ابزارهای متفاوتی برای تهیه کامپایلر استفاده می‌کنند. برخی از این ابزارها به شرح زیر اند.

۱. تولید کننده تحلیل‌گر لغوی¹ به صورت خودکار با استفاده از یک توصیف از توکن‌های زبان در زبان منظم، تحلیل‌گر لغوی تهیه می‌کند.
۲. تولید کننده تجزیه کننده² به صورت خودکار یک تحلیل‌گر نحوی از یک توصیف گرامری تهیه می‌کند.
۳. مترجم نحوی³ با استفاده از یک توصیف معنایی از قوانین گرامر، و مجموعه‌ای از قواعد برای پیمایش درخت تجزیه، یک تحلیل‌گر معنایی و تولیدکننده کد میانی تهیه می‌کند.

¹ scanner generator

² parser generator

³ syntax-directed translator

۴. تولید کننده تولید کننده کد⁴ : با استفاده از مجموعه‌ای از قوانین ترجمه دستورات کد میانی به کد ماشین، یک تولید کننده کد تهیه می‌کند.
۵. موتور تحلیل جریان داده⁵ : تحلیل جریان داده یکی از بخش‌های مهم بهینه‌ساز کد است. موتور تحلیل جریان داده با جمع‌آوری اطلاعات در مورد اینکه بخش‌های مختلف برنامه چگونه با یکدیگر ارتباط برقرار می‌کنند، کد تولید شده را بهینه‌سازی می‌کند.
۶. ابزار تولید کامپایلر⁶ : مجموعه‌ای از توابع و ابزارها برای ساخت کامپایلر ارائه می‌کند.

⁴ code-generator generator

⁵ data-flow analyzer

⁶ compiler-construction toolkit

- کامپیوترهای الکترونیکی ابتدایی در دههٔ ۱۹۴۰ به وجود آمدند. برنامه‌های آنها به زبان ماشین یعنی با استفاده از صفر و یک‌ها توصیف می‌شدند. با استفاده از ورودی‌های دودویی به کامپیوتر دستور داده می‌شد که عملیات خواندن از حافظه، جمع، ضرب، نوشتن بر روی حافظه و غیره را انجام دهد. نیاز به گفتن نیست که این نوع برنامه‌نویسی بسیار پیچیده است و احتمال ایجاد خطا در آن زیاد است. همچنین هنگامی که برنامه‌ای تولید شد، خواندن و فهمیدن و تغییر دادن آن بسیار مشکل است.

- اولین گام در طراحی زبان‌هایی که به زبان انسان شباهت بیشتری داشته باشند تا زبان ماشین، در طراحی زبان اسمبلی در اوایل دههٔ ۱۹۵۰ صورت گرفت. در زبان اسمبلی دستورات زبان ماشین که با صفر و یک بیان می‌شوند، توسط کلمات معنی‌دار زبان انگلیسی جایگزین شدند.
- در اواخر دهه ۱۹۵۰ زبان فورترن^۱ برای انجام محاسبات علمی، زبان کوبول^۲ برای انجام پردازش داده در کاربردها تجاری، و زبان لیسپ^۳ برای انجام محاسبات بر روی لیست‌ها و کاربردهای هوش مصنوعی توسعه یافتند. دلیل ابداع این زبان‌ها، ایجاد امکان توصیف برنامه‌ها به زبانی بود که به زبان انسان‌ها و زبان ریاضی شباهت بیشتری داشته باشد. برای مثال زبان فورترن برای تبدیل فرمول‌های ریاضی به زبان اسمبلی وجود آمد تا با استفاده از آن برنامه‌نویسان بتوانند فرمول‌ها را شبیه به توصیف ریاضی آنها در برنامه توصیف کنند.

^۱ Fortran

^۲ Cobol

^۳ Lisp

- زبان‌های برنامه‌نویسی را می‌توان براساس رویکرد آنها به توصیف برنامه‌ها دسته‌بندی کرد. یک دسته از زبان‌های برنامه‌نویسی که زبان‌های دستوری¹ نامیده می‌شوند، در توصیف یک برنامه به چگونگی نحوه اجرای برنامه می‌پردازند. به عبارت دیگر یک برنامه به زبان دستوری توصیف می‌کند چگونه محاسبات انجام می‌شوند. دسته‌ای دیگر از زبان‌ها، زبان‌های اعلامی² نامیده می‌شوند. یک برنامه به زبان اعلامی مشخص می‌کند چه محاسباتی باید انجام شود و چگونگی انجام محاسبات غالباً بر عهده کامپایلر است.
- زبان‌های سی، سی++ و جاوا در دسته زبان‌های دستوری قرار می‌گیرند و زبان‌های لیسپ، ام‌ال، هسکل و پرولوگ در دسته زبان‌های اعلامی.

¹ imperative

² declarative

- همچنین می‌توان زبان دستوری را به دو دسته زبان‌های رویه‌ای¹ و زبان‌های شیء‌گرا² تقسیم کرد. یک برنامه در یک زبان رویه‌ای شامل تعداد تابع است که مقادیر خروجی خود را به یکدیگر منتقل می‌کنند و یک برنامه در یک زبان شیء‌گرا از تعدادی شیء تشکیل شده که هر کدام ویژگی و رفتارهایی دارند و از طریق رفتارها با یکدیگر در ارتباط‌اند. سی یک زبان رویه‌ای و سی++ و جاوا مثال‌هایی از زبان‌های شیء‌گرا هستند.
- زبان‌های اعلامی را نیز می‌توان به چند دسته تقسیم کرد. زبان‌های تابعی³ زبان‌هایی هستند که یک برنامه را با استفاده از تعدادی تابع همانند توابع ریاضی توصیف می‌کنند و زبان‌های منطقی⁴ زبان‌هایی هستند که یک برنامه را با استفاده از تعدادی گزاره منطقی توصیف می‌کنند. لیسپ و اسکیم و هسکل و ام‌ال در دسته زبان‌های تابعی، و پرولوگ در دسته زبان‌های منطقی قرار می‌گیرند.

¹ procedural

² object-oriented

³ functional

⁴ logical

- طراحان کامپایلر باید از طرفی با ویژگی‌های زبان‌های جدید و نیازهای جدید زبان‌های برنامه‌نویسی آشنا باشند و از طرفی دیگر قابلیت‌های ماشین‌ها و سخت‌افزارهای جدید را بشناسند.
- موضوع علم کامپایلر این است که چگونه الگوریتم‌های مناسب را برای پیاده‌سازی کامپایلر یک زبان به کار ببریم به طوری که از طرفی الگوریتم‌های طراحی شده عمومی باشند و همه برنامه‌های ورودی را در برگیرند و از طرفی کارآمدی بالایی داشته باشد. بنابراین در طراحی کامپایلر به علم طراحی الگوریتم و ساختمان داده و معماری ماشین نیاز است.
- همچنین از علم نظریه زبان‌های و ماشین‌ها در کامپایلر استفاده می‌شود، چراکه ماشین‌های متناهی و گرامرهای منظم برای توصیف واژگان زبان و گرامرهای مستقل از متن و الگوریتم‌های مربوط به آنها برای توصیف ساختار زبان و پیاده‌سازی آن مورد استفاده قرار گیرند. در پیاده‌سازی یک کامپایلر اصول و قوانین طراحی نرم‌افزار نیز باید رعایت شوند، بنابراین از علم طراحی نرم‌افزار نیز استفاده می‌شود.